CPro’26 Midsem Rubric

Q1 - 3 Marks

```
int i = 0, j = 0;

while (i < 2) {
    while (j < 2) {
        printf("%d %d ", i, j);
        j++;
    }
    i++;
}
```

Explanation - `j` is **not reset to 0** after the inner loop finishes. Then `i` becomes `1`, but the condition `j < 2` is already false, so the inner loop does not execute again. Correct output: `0 0 0 1`

[1] for `0 0`

[3] for complete correct output `0 0 0 1`

[0] for anything else

Q2 - 2 Marks

```
char a[] = "Hello";
char b[] = "Hello";

if (a == b)
    printf("Equal");
else
    printf("Not Equal");
```

Explanation - `a` and `b` are two different character arrays. When used in this comparison, they refer to the addresses of the two separate arrays. Their contents are equal, but their addresses are different.

[1] for understanding that `a == b` compares the addresses/references of the arrays, not their contents

[2] for `Not Equal`

[0] for anything else

Q3 - 2 Marks

int j = _____

Explanation - At every iteration, the valid indices in the remaining unshuffled portion are `0` through `i`, inclusive.

Therefore:

int j = rand() % (i+1);

[2] for `rand() % (i + 1)` or a similar expression

[0] for anything else

Q4 - 3 Marks

int n = 4;

```
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= ____(A)____; j++){
        printf("* ");
    }
    ____(B)____;
}
```

Explanation - For a decreasing triangle, the number of stars should be: `4,3,2,1` for `i = 1, 2, 3, 4`.

Therefore:

A=n-i+1
B=printf("\n");

[1.5] for `A = n - i + 1`

[1.5] for `B = printf("\n");`

[3] for both correct blanks

[0] for anything else

Q5 - 3 Marks

int a[] = {12, 5, 18, 7, 15, 10};
int i, n = 6;

```
int largest = a[0];
int second = a[0];

for (i = 1; i < n; i++){
    if (a[i] > largest){
        second = _____;
        largest = _____;
    }
    else if (a[i] > second){
        second = _____;
    }
}
```

Explanation - When a new largest element is found, the previous largest becomes the second largest.

Therefore:

```
second=largest;
largest=a[i];
```

If the element is not larger than `largest` but is larger than `second`, it becomes the new second largest: `second=a[i];`

[1] for `second = largest`

[+1] for `largest = a[i]`

[+1] for `second = a[i]`

[3] for all three correct blanks

[0] for anything else

Q6 - 3 Marks

```
void change(int x) {
    x = x + 10;
    printf("%d ", x);
}

int main() {
    int x = 5;
    change(x);
    printf("%d", x);
}
```

Explanation - C passes the integer `x` **by value**. Therefore, `change()` receives a copy of `x`.

Inside the function: `x = 5 + 10 = 15`

so `15` is printed. The original `x` in `main` remains `5`. Correct output: `15 5`

[1] for `15`

[+1] for `5`

[3] for complete correct output `15 5` with space

[0] for anything else

Q7 - 3 Marks

```
int f(int x) {
    if (x % 2 == 0)
        return x / 2;
    else
        return x + 1;
}

int a = 7;
a = f(a);
printf("%d ", a);
a = f(a);
printf("%d", a);
```

Since `7` is odd: `f(7) = 7 + 1 = 8`

Therefore: `a = 8`

Next: `f(8) = 8 / 2 = 4`

Therefore: `a = 4`

Correct output: `8 4`

[1] for `8`

[+1] for `4`

[3] for complete correct output `8 4`

[0] for anything else

Q8 - 2 Marks

```
char s[10] = "hello";

s[2] = '\0';

printf("%s", s);
```

Explanation - The string initially contains:

h e l l o \0
0 1 2 3 4 5

Setting `s[2]` to `'\0'` terminates the string after `he`.

Therefore: `he`

[1] for understanding that `'\0'` terminates the string

[2] for `he`

[0] for anything else

Q9 - 4 Marks

```
int a[] = {5, 6, 7, 8};
int i, x = 0;

for (i = 0; i < 4; i++) {
    if (a[i] & 1)
        x = x | (1 << i);
    else
        x = x & ~(1 << i);

    printf("%d\n", x);
}
```

Explanation - The program updates bit `i` of `x` based on whether `a[i]` is odd or even, and `printf` is executed, so `x` is printed after every iteration.

Array values:

a[0] = 5 → odd → set bit 0 → x = 0001_2 = 1
a[1] = 6 → even → clear bit 1 → x = 0001_2 = 1
a[2] = 7 → odd → set bit 2 → x = 0101_2 = 5
a[3] = 8 → even → clear bit 3 → x = 0101_2 = 5

Therefore, the program prints:

1
1
5
5

[1] for correctly identifying that odd array elements set the corresponding bit

[+1] for correctly identifying that even array elements clear the corresponding bit

[+1] for correctly determining the value of `x` after each iteration as `1, 1, 5, 5`

[+1] for fully correct answer with the output in newline

[0] for anything else

Q10 - 4 Marks

Valid approach -

```
char s[1000];
int freq[128] = {0};

fgets(s, sizeof(s), stdin);

for (int i = 0; s[i] != '\0'; i++) {
    if (s[i] != '\n')
        freq[(unsigned char)s[i]]++;
}

int max = 0;
char ans;

for (int i = 0; i < 128; i++) {
    if (freq[i] > max) {
        max = freq[i];
        ans = (char)i;
    }
}

printf("%c", ans);
```

[1] for correctly reading a complete line of text

[+1] for maintaining a frequency count for ASCII characters

[+1] for finding the character with maximum frequency

[+1] for correctly printing/returning the most frequent character

[0] for fundamentally incorrect logic

Q11 - 4 Marks

Valid appraoch -

```
int isPalindrome(int n) {
    int original = n;
    int reverse = 0;

    while (n > 0) {
        int digit = n % 10;
        reverse = reverse * 10 + digit;
        n = n / 10;
    }

    return original == reverse;
}
```

Explanation - The digits are extracted from right to left using `% 10`, and the reversed number is constructed using:

reverse = reverse*10 + digit;

Finally, the reversed number is compared with the original number.

[1] for correctly extracting digits using `% 10`

[+1] for correctly removing digits using `/ 10`

[+1] for correctly constructing the reversed number

[+1] for comparing the reversed number with the original number and returning the correct result

[-1] if a function is not written with the correct datatype for argument and return value

Accept equivalent correct implementations, including implementations that convert the number to a string and check the string from both ends, provided the implementation correctly handles the stated positive-integer requirement.

Q12 - 4 Marks

Valid approach-

```
void rotateRight(int a[], int n) {
    if (n <= 2)
        return;

    int x = a[n - 2];
    int y = a[n - 1];

    for (int i = n - 1; i >= 2; i--) {
        a[i] = a[i - 2];
    }

    a[0] = x;
    a[1] = y;
}
```

[1] for correctly storing the last two elements before shifting

[+1] for correctly shifting the remaining elements by 2 positions in reverse order

[+1] for placing the stored elements at `a[0]` and `a[1]`

[+1] for correct in-place solution that works for general `n`

[-1] for not accounting `n ≤ 2`

[0] for anything else

Accept equivalent correct implementations, including implementations that use other in-place approaches to rotate the array right by two positions, provided they work correctly for an array of any size `n` and do not use another array, like reversal or recursion.

Q13 - 5 Marks

Explanation - **Insertion Sort** will perform better for this nearly sorted array. Insertion Sort only needs to move elements that are out of their correct position, whereas Bubble Sort repeatedly compares adjacent elements across multiple passes.

Two valid mechanical reasons:

1. **Fewer comparisons:** Insertion Sort stops comparing once the current element reaches its correct position, whereas Bubble Sort continues making adjacent comparisons across each pass.
2. **Fewer/more efficient data movements:** Insertion Sort shifts elements only when necessary. Bubble Sort performs swaps, and each swap involves multiple assignments.

For the given array:

[2, 5, 7, 12, 10, 15]

Bubble Sort

Using standard Bubble Sort without an early-termination optimization:

Pass 1 → 5 comparisons, 1 swap
Pass 2 → 4 comparisons, 0 swaps
Pass 3 → 3 comparisons, 0 swaps
Pass 4 → 2 comparisons, 0 swaps
Pass 5 → 1 comparison, 0 swaps

Therefore:

15 comparisons
1 swap

Insertion Sort

Insert 5 → 1 comparison, 0 shifts
Insert 7 → 1 comparison, 0 shifts
Insert 12 → 1 comparison, 0 shifts
Insert 10 → 2 comparisons, 1 shift
Insert 15 → 1 comparison, 0 shifts

Therefore:

6 comparisons
1 shift

[1] for identifying **Insertion Sort**

[+1] for giving **two valid mechanical reasons** explaining why Insertion Sort is better suited to the nearly sorted array

[+1.5] for correctly showing Bubble Sort as **15 comparisons or 9 comparisons (with optimized version) and 1 swap**

[+1.5] for correctly showing Insertion Sort as **6 comparisons and 1 shift**

[0] for anything else